Architecture and Trade-offs Document

In-M

Author: Sandeep Badikolu
GitHub: https://github.com/sandeep-badikolu/InMemoryAccountLedger
Purpose: Part 2 submission and preparation for 45-minute live technical defense.

This document answers the four architecture prompts from the Staff Software Engineer assessment. Part 1 implements a deterministic replay engine over a six-day event stream (ACC-001 AED, ACC-002 BHD) with append-only semantics, authorization holds, fee and interest rules, and explicit rejection of invalid settlements (E6 / Auth-Z).

1. Append-only design at scale

1.1 What breaks at 100x event volume?

The reference implementation materializes the full input stream in memory and, for each reporting day D, recomputes closing ledger balance by scanning all accepted events with value_date <= D. Holds, fees, and interest accruals repeat similar scans. Complexity per full replay is approximately O(days x events x accounts). At 100x volume (millions of events instead of tens), three pain points appear quickly.

First, latency: batch end-of-day and customer balance APIs cannot afford a full historical scan on every request. Second, memory: the event log, rejection log, and derived system events (overdraft fees, interest capitalization) grow without an upper bound in this design. Third, operational fragility: a single hot account with a long history dominates scan time; tail latency spikes even if median accounts are small.

The exercise is correct pedagogically: append-only event sourcing is the right long-term model, but naive replay-from-scratch does not survive production scale without structural supporting artifacts.

1.2 Where is state unbounded?

Unbounded or linearly growing state includes: (a) the immutable event log (customer and system events); (b) authorization metadata for every distinct auth id ever created; (c) optional materialized daily snapshots if cached without a retention policy; (d) audit copies of rejected events (e.g. E6) retained for compliance. None of these are compacted in the exercise. Even bounded-domain data such as active holds can become large if authorizations expire slowly and TTL enforcement is missing (Auth-B in the assignment remains open through Day 6).

1.3 Cheapest structural change that preserves append-only semantics

Introduce time-bucketed segments with immutable boundary snapshots. Example: at each calendar day close, append a segment header recording per-account ledger balance, sum of active holds, and last applied sequence number. Replay for date T loads the latest snapshot with snapshot_date < T plus only events in the tail bucket. Historical events are never updated or deleted; new information is always appended (segment headers are events too).

This is the lowest-cost change because it does not require a relational schema redesign or abandoning event sourcing. It mirrors how banks already think in posting periods. Follow-on work (not required for the exercise) would add account-level indexes, incremental projections for statements, and cold storage tiering for segments older than N years.

2. Value-dated entries in a UAE-licensed bank

2.1 Operational and regulatory surface

Value date is the accounting date on which an entry affects balances; recorded date is when the event entered the processing pipeline. They diverge in the assignment when E7 debits ACC-001 with value_date Day 2 but is recorded on Day 5, temporarily moving the Day 2 close to AED -570 before E9 reverses E7 on Day 6.

For a UAE-licensed institution this touches multiple surfaces. Customer channels must show balances and holds consistent with the authoritative ledger as-of the disclosed business date. Finance must align sub-ledger balances to the general ledger and explain any backdated adjustments in management reporting. Compliance and AML teams require immutable evidence of who introduced a backdated movement, the business justification, and whether customer advice was required. Regulatory returns that depend on end-of-day balances (liquidity, large exposure context, conduct metrics) can be restated if value dating is handled informally.

2.2 One control mandated before production go-live

Effective-date governance with maker-checker and automated restatement.

Policy: any posting with value_date earlier than the current business date is blocked in straight-through processing and routed to a controlled queue. A second approver (operations or finance control, segregated from the maker) must approve with a reason code and customer-impact classification. The system stores approver identity, timestamp, and linked case id in an append-only audit log.

Upon approval, a restatement job deterministically recomputes affected days: closing balances, overdraft fees, interest accruals, and downstream regulatory extracts. Original events remain immutable; corrections arrive as compensating entries or explicitly linked adjustment events. Customers receive amended statements when material. This single control reduces the risk that backdating (as in E7) silently corrupts historical closes while still honoring the append-only principle required by the assignment.

3. Authorization lifecycle beyond settlement

The assignment models settlement (E5) as the happy path that debits the ledger and clears a hold. Production card and account-based payments require additional terminal states and transitions.

3.1 Expiry (time-to-live)

Scenario: hotel or car-rental pre-authorization. The merchant never captures the full hold. Mandated behavior: at TTL, automatically release the hold without ledger debit; emit an expiry event for audit; update available balance on channels. Stale holds must appear in operations dashboards (Auth-B in the exercise is an open hold through Day 6).

3.2 Partial capture

Scenario: fuel pump pre-auth for AED 200, final charge AED 185 (similar to Auth-A). Mandated behavior: post settlement for actual amount; release residual hold; never debit more than authorized unless a new authorization is obtained.

3.3 Void and reversal before capture

Scenario: merchant cancels before batch capture. Mandated behavior: idempotent void keyed by authorization id; remove hold; no duplicate void side effects on retry.

3.4 Incremental and linked authorizations

Scenario: restaurant tip adjustment or multi-shipment e-commerce. Mandated behavior: support incremental auth amounts with a parent/child link and a hard cap on total exposure.

3.5 Post-capture disputes (chargebacks)

Scenario: customer disputes a settled transaction. Mandated behavior: do not reuse the original authorization id as a settlement; open a dispute case with its own ledger postings after investigation. Settlement and dispute are separate lifecycles.

4. Scope trade-offs in the exercise implementation

The codebase deliberately limits scope to meet the 48-hour exercise while remaining defensible in a live defense. Each simplification trades delivery speed for deferred production risk.

4.1 Persistence and availability

Cut: no database, no replication, no disaster recovery. Why: assignment forbids external persistence. Deferred risk: total data loss on process crash; no multi-region continuity; no point-in-time recovery for regulators.

4.2 Concurrency and integration

Cut: single-process replay; no HTTP API; no authentication. Why: focus on ledger rules and determinism. Deferred risk: race conditions if multiple writers update the same account; no standard integration path for channels; no rate limiting or fraud scoring hooks.

4.3 Fees, interest, and policy conflicts

Cut: fees recomputed from the final value-dated ledger after the full stream (see REJECTED.md). Why: avoids modeling separate fee-reversal events within the exercise window. Deferred risk: point-in-time fee assessments may differ from operational truth; the included intentionally failing test documents the conflict between E7-driven Day-2 fees and post-E9 rollback expectations.

4.4 Authorization availability rules

Cut: simplified rule using recorded day and value day so Auth-B can be approved against the Day 4 close despite backdated E7 (documented in AMBIGUITIES.md section 9). Why: reconcile assignment narrative with stream ordering. Deferred risk: available balance on channels may not match card-network real-time authorization unless rules are unified.

4.5 Idempotency, FX, and product features

Cut: idempotency beyond unique event ids; no FX; interest capitalized only on Day 6; BHD instalments split manually (3.333 + 3.333 + 3.334). Why: spec boundaries and numeric precision demonstration. Deferred risk: duplicate message delivery could double-post; cross-currency payments unsupported; no mid-period interest payouts or tax withholding.

5. Defense alignment with Part 1 artifacts

Reviewers can trace arguments to repository files: NUMBERS.md (constants), AMBIGUITIES.md (value dating, auth balance, instalment rounding), REJECTED.md (refused acceptance criteria such as three x BHD 3.334 and discarding interest remainder), WORKLOG.md (delivery history), and FAILING_TEST.md with inline comments in LedgerEngineTests. Running dotnet test shows twelve passing tests and one intentional failure that encodes the fee versus reversal policy debate.

Production evolution path: external append-only log (e.g. Kafka plus object storage), per-account command serialization with optimistic locking, materialized read models for statements, explicit fee and interest entries with reversal policy, and authorization TTL with regulatory-grade audit.

5.1 Event stream anchors for oral defense

E6 (Auth-Z, AED 160): rejected - no prior authorization; funds must not leave via orphan settlement. E5 (Auth-A, AED 185): accepted partial capture against hold AED 200. E7/E9: backdated debit AED 820 and compensating reversal on value_date Day 2; original debit event remains immutable. E8 (Auth-B): hold reduces available balance, not ledger balance; never settled in window. E10: BHD 10.000 as 3.333 + 3.333 + 3.334 (reject criterion that each instalment is 3.334 because sum would exceed 10.000).

Daily outputs from Ledger.Runner: closing ledger, holds, available, fees assessed, active authorizations, and errors by recorded day - matching Part 1 deliverable expectations.

5.2 Likely live-defense questions and concise answers

Why recompute fees after the full stream? To keep append-only input events and avoid undefined fee-reversal policy; document the trade-off via the failing test.

How would you test concurrency? Serialize commands per account; property-based tests on balance monotonicity under parallel debits; chaos tests on duplicate delivery with idempotency keys.

What is the first production service you would add? An ingestion API that validates, assigns monotonic sequence numbers, and appends to the durable log before acknowledging clients - replay remains the correctness oracle.